

Navigation Package

A* Path Planning on Occupancy Grids for ORB-SLAM3 Maps

ORB-SLAM3 Relocalization Project

May 2026

Contents

1	Introduction	3
2	System Architecture	3
2.1	Node Roles	4
3	Occupancy Grid Construction	4
3.1	Pipeline Overview	4
3.2	Step 1 — Height Filter	4
3.3	Step 2 — Density Filter	4
3.4	Step 3 — Grid Projection	5
3.5	Step 4 — Smoothing	5
4	Coordinate Systems	5
4.1	Conversions	5
5	Obstacle Inflation	6
5.1	Implementation	6
6	A* Path Planning	7
6.1	Cost Functions	7
6.2	Algorithm — Open List with Lazy Deletion	8
6.3	Template Method Pattern	9
7	Path Smoothing	9
7.1	Adaptive Waypoint Downsampling	9
7.2	Natural Cubic Spline	10
8	Pose Sources	10
8.1	Dummy Mode	10
8.2	Relocalization Mode	11
9	Deviation Detection and Replanning	11
10	Visualization Pipeline	11
10.1	Trail	12
10.2	Heading Indicator (Triangle)	12

11 Configuration	12
12 End-to-End Flowchart	13
13 Key Takeaways	13

1 Introduction

The **navigation package** turns a 3-D ORB-SLAM3 point cloud into a 2-D occupancy grid and plans collision-free paths across it using A*. It is designed as a standalone ROS2 Humble Python package that plugs directly into the rest of the relocalization pipeline: the same map that ORB-SLAM3 uses for tracking is projected top-down, and the live pose from `/relocalization/pose` drives the robot indicator on the map.

The package solves three problems:

1. **Map building** — converting sparse 3-D point cloud data (exported as a CSV) into a clean 2-D binary occupancy grid.
2. **Path planning** — finding a smooth, obstacle-free path between the robot's current position and a user-selected goal using A*.
3. **Visualization** — displaying the grid, the planned path, and the robot's trail in real time via a PyQt5 window.

Note

You do *not* need to understand ORB-SLAM3 internals to use this package. The only input it needs is a CSV of 3-D map points (x, y, z columns) and, optionally, a live `geometry_msgs/Pose` from the relocalization node.

2 System Architecture

The package contains three ROS2 nodes that communicate over topics.

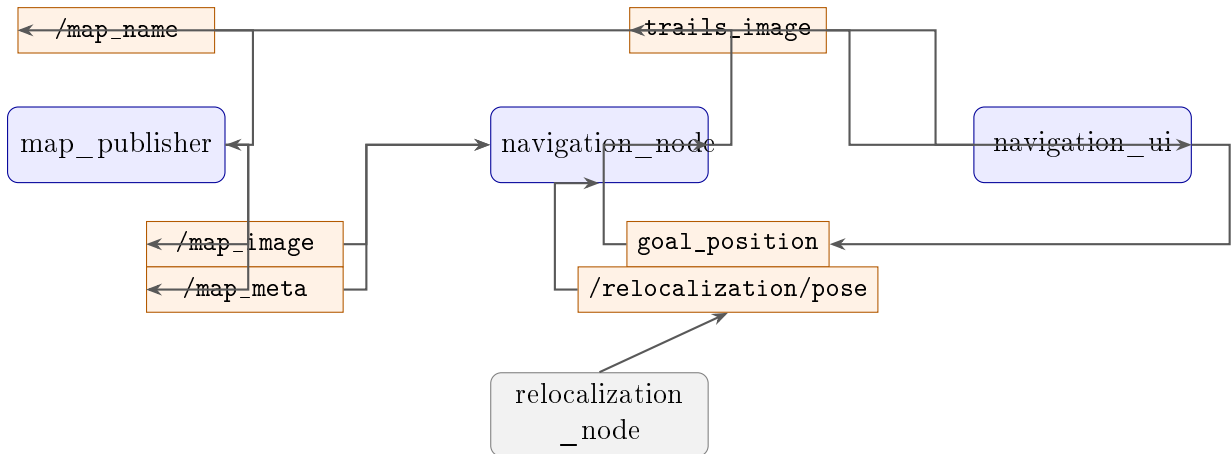


Figure 1: ROS2 topic graph of the navigation package. Arrows show data flow direction.

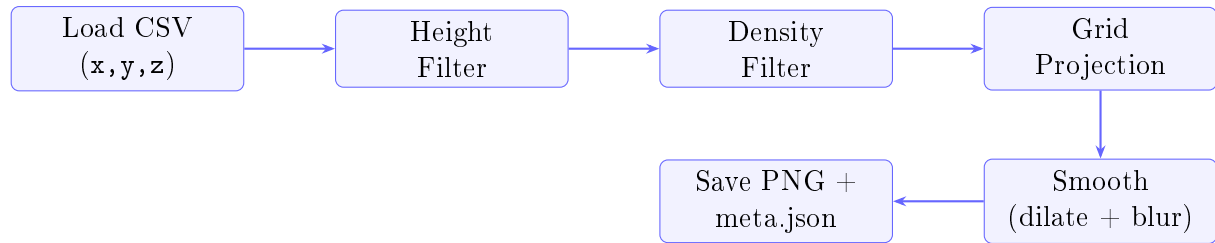
2.1 Node Roles

Node	Executable	Role
map_publisher	ros2 run navigation map_publisher	Loads CSV/PNG map, publishes grid a
navigation_node	ros2 run navigation navigation	A* planning, deviation check, visualiza
navigation_ui	ros2 run navigation navigation_ui	PyQt5 window, goal clicks, map selecto

3 Occupancy Grid Construction

An **occupancy grid** is a 2-D array where each cell holds a binary value: 1 (occupied / wall) or 0 (free / traversable). The `map_publisher` node builds this grid from a CSV of 3-D map points exported from ORB-SLAM3.

3.1 Pipeline Overview



3.2 Step 1 — Height Filter

ORB-SLAM3 maps contain points at all heights: floor, walls, ceiling, furniture. For 2-D navigation we keep only points near the “waist” of the scene.

Height Filter

Let μ_y and σ_y be the mean and standard deviation of all y -coordinates (vertical axis in ORB-SLAM3 convention). Keep point p if:

$$\mu_y - \sigma_y < p_y < \mu_y + \sigma_y$$

This removes floor and ceiling points while keeping the bulk of wall structure.

3.3 Step 2 — Density Filter

Sparse “stray” points from reflections or tracking errors are removed using a neighbourhood count:

Density Filter

Build a BallTree over the (x, z) projections. Keep point p if:

$$|\{q \mid \|q_{xz} - p_{xz}\| < 0.3 \text{ m}\}| \geq 3$$

Points with fewer than 3 neighbours within 30 cm are discarded as outliers.

3.4 Step 3 — Grid Projection

The surviving points are projected onto a 2-D grid. ORB-SLAM3 uses **right-handed coordinates**: X rightward, Y upward, Z forward. For top-down navigation we project onto the X - Z plane.

World-to-Grid Conversion

Given resolution r (metres per pixel) and minimum world coordinates $(x_{\min}, z_{\min})$:

$$\text{col} = \left\lfloor \frac{x - x_{\min}}{r} \right\rfloor \quad (1)$$

$$\text{row} = \left\lfloor \frac{z - z_{\min}}{r} \right\rfloor \quad (2)$$

The default resolution is $r = 0.02$ m/px (50 pixels per metre). Grid cells that contain at least one projected point are marked 255 (wall).

Note

Subtracting $(x_{\min}, z_{\min})$ shifts the coordinate origin to the map corner, so pixel $(0, 0)$ corresponds to the world minimum, *not* the ORB-SLAM3 world origin $(0, 0, 0)$. This offset is stored in `map.meta.json` and broadcast on `/map_meta` so the navigation node can convert live poses correctly.

3.5 Step 4 — Smoothing

Raw point-cloud grids are noisy. A two-stage smoothing closes small gaps and merges nearby wall clusters:

1. **Morphological dilation** with a 7×7 kernel, 2 iterations — thickens wall pixels outward.
2. **Gaussian blur** (9×9 , $\sigma = 2$) — blends edges.
3. **Re-threshold** at 127 — restores binary values.

The resulting PNG is saved alongside the CSV and reloaded on subsequent runs, skipping the expensive CSV pipeline entirely.

4 Coordinate Systems

Three coordinate systems appear in this package. Confusing them is the most common source of bugs.

4.1 Conversions

Let H be the grid height in pixels and $r = 0.02$ m/px.

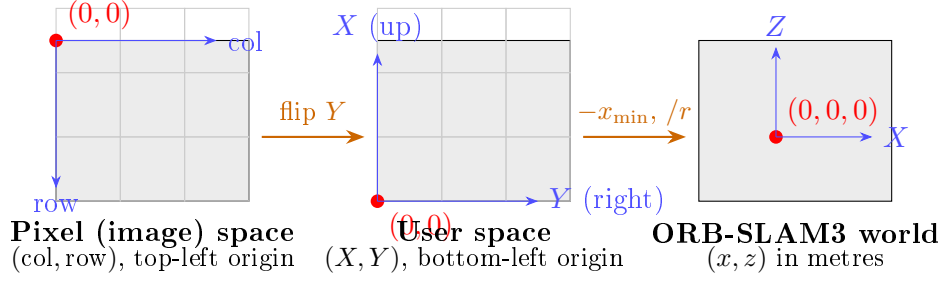


Figure 2: The three coordinate systems. The navigation node converts *into* pixel space for all internal computation.

User Space $\rightarrow$ Pixel Space

The UI publishes goals in user space (X_u, Y_u) with origin at bottom-left, X upward:

$$\text{col} = Y_u \quad (3)$$

$$\text{row} = H - 1 - X_u \quad (4)$$

ORB-SLAM3 World $\rightarrow$ Pixel Space

The relocalization node publishes `geometry_msgs/Pose` with position (p_x, p_y, p_z) in ORB-SLAM3 world metres:

$$\text{col} = \left\lfloor \frac{p_x - x_{\min}}{r} \right\rfloor \quad (5)$$

$$\text{row} = H - 1 - \left\lfloor \frac{p_z - z_{\min}}{r} \right\rfloor \quad (6)$$

The subtraction of $(x_{\min}, z_{\min})$ is essential: without it, the robot at world position $(0,0)$ would be placed at a pixel far outside the actual map extent.

Note

The row formula contains $H - 1 - \dots$ because increasing Z (moving forward in ORB-SLAM3) should move the robot *upward* on screen. Image rows increase downward, so we invert the row index.

5 Obstacle Inflation

A robot is not a point — it has physical width. Even if the robot centre follows a path that technically avoids obstacles, the robot body may still collide with a wall. **Obstacle inflation** solves this by expanding every obstacle cell outward before planning, so the planner treats the robot as a point but the inflated walls already include a safety margin.

5.1 Implementation

Inflation is computed *once per map update*, not once per planning call:

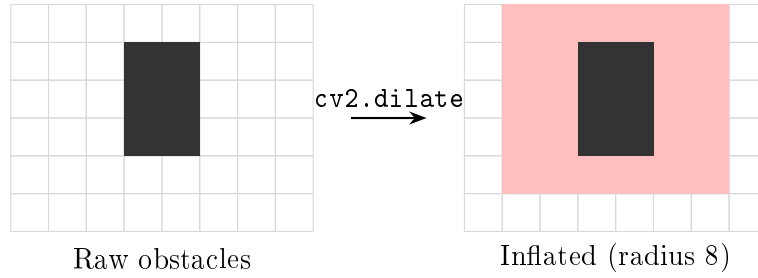


Figure 3: Obstacle inflation: original wall cells (black) are expanded into a forbidden zone (red). The planner plans only in white cells.

Listing 1: Inflation in `map_callback` (`main.py`)

```

1 # kernel covers all 8 directions equally --- MORPH_RECT preserves
  corners
2 kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (2*radius+1, 2*
  radius+1))
3 inflated = cv2.dilate(binary_map, kernel, iterations=1)

```

Note

Why MORPH_RECT and not MORPH_ELLIPSE? A rectangular kernel inflates corners diagonally, matching 8-connected A* movement. An elliptical kernel leaves diagonal corners exposed, which can allow the planner to route through a one-pixel gap at a wall corner.

The cached inflated map is reused for every plan call. At plan time, the start and goal cells are explicitly cleared (set to 0) so they are never treated as obstacles even if they lie inside the inflation zone.

6 A* Path Planning

A* is a best-first graph search algorithm that finds the optimal (lowest-cost) path between two cells in a grid. It combines the actual cost to reach a node with an estimate of the remaining cost to the goal.

6.1 Cost Functions

A* Scores

For each grid cell n :

$$g(n) = \text{actual cost from start to } n \quad (7)$$

$$h(n) = \text{estimated cost from } n \text{ to goal (heuristic)} \quad (8)$$

$$f(n) = g(n) + h(n) \quad (9)$$

A* always expands the open node with the lowest f -score.

This implementation uses **Euclidean distance** as the heuristic, which is admissible (never overestimates) because straight-line distance is always $\leq$ the true path length:

Heuristic

$$h(n) = \sqrt{(n_x - g_x)^2 + (n_y - g_y)^2}$$

where (g_x, g_y) is the goal cell in pixel coordinates.

Movement cost. The planner allows 8-connected movement (cardinal and diagonal). Moving diagonally costs $\sqrt{2}$ times a cardinal step to avoid artificially preferring diagonals:

Step Cost

$$c(n, m) = \begin{cases} 1 & \text{if } m \text{ is a cardinal neighbour of } n \\ \sqrt{2} & \text{if } m \text{ is a diagonal neighbour of } n \end{cases}$$
$$g(m) = g(n) + c(n, m)$$

6.2 Algorithm — Open List with Lazy Deletion

The classic A* maintains two sets:

- **Open set** — nodes discovered but not yet expanded (a priority queue ordered by f).
- **Closed set** — nodes already expanded.

A naive open set backed by a list requires $O(n)$ to find and update the minimum- f entry. This implementation uses a **binary heap** (heapq) with **lazy deletion**:

Lazy Deletion

When a node's g -cost improves, a *new* heap entry is pushed with the updated f . The stale entry remains in the heap but is **skipped** at pop time by checking whether the node was already visited. This avoids costly $O(n)$ heap updates while keeping $O(\log n)$ push and pop.

Algorithm 1 A* with Heapq and Lazy Deletion

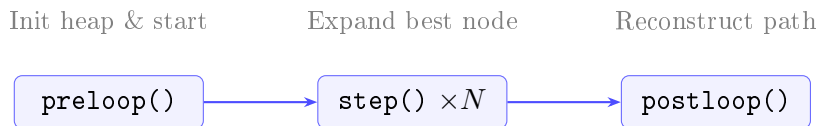
```
1: Push ( $f_{\text{start}}$ , counter, start) onto heap;  $g[\text{start}] \leftarrow 0$ 
2: while heap is not empty and iterations < limit do
3:   ( $\_$ ,  $\_$ ,  $n$ )  $\leftarrow$  POP(heap)
4:   if  $n \in \text{visited}$  then continue  $\triangleright$  lazy deletion — stale entry
5:   end if
6:   Add  $n$  to visited
7:   if  $\|n - \text{goal}\| \leq 1.5$  and line-of-sight to goal is clear then
8:     goal.parent  $\leftarrow n$ ; break
9:   end if
10:  for each free neighbour  $m$  of  $n$  do
11:     $g_{\text{new}} \leftarrow g[n] + c(n, m)$ 
12:    if  $g_{\text{new}} < g[m]$  then
13:       $g[m] \leftarrow g_{\text{new}}$ 
14:       $f \leftarrow g_{\text{new}} + h(m)$ 
15:      Push ( $f$ , counter+1,  $m$ ) onto heap; set  $m.\text{parent} \leftarrow n$ 
16:    end if
17:  end for
18: end while
19: return path reconstructed by following parent pointers from goal to start
```

Note

The **counter** tiebreaker in the heap tuple (f , counter, n) ensures that two nodes with equal f are never compared directly (which would require `PathNode.__lt__` to impose an arbitrary ordering). The integer counter is always unique, so `heapq` never falls through to compare the node objects.

6.3 Template Method Pattern

The planner is structured as an abstract base class with three hooks:



The `plan()` method in the base class calls these three hooks in order up to `iter_limit = 10000` iterations. `AStarImplementation` overrides all three.

7 Path Smoothing

The raw A* path is jagged — it follows grid diagonals and cardinal steps. A smooth path is easier for a robot to follow and nicer to visualise.

7.1 Adaptive Waypoint Downsampling

The full A* path may contain hundreds of nodes. Before fitting a spline, the path is downsampled to a small set of **representative waypoints**:

- Corners (high curvature turns) — kept at dense spacing.
- Straight segments — kept at sparse spacing.

This balances faithfulness to the original path near obstacles with smoothness on open stretches.

7.2 Natural Cubic Spline

A **natural cubic spline** is fit independently to the x and y coordinates of the waypoints, parameterised by cumulative arc length s :

Cubic Spline Parameterisation

Given waypoints $(x_0, y_0), (x_1, y_1), \dots, (x_k, y_k)$ with cumulative arc lengths $s_0 = 0, s_i = s_{i-1} + \|(x_i - x_{i-1}, y_i - y_{i-1})\|$:

$$x(s) = \sum_i a_i^x s^3 + b_i^x s^2 + c_i^x s + d_i^x \quad (10)$$

$$y(s) = \sum_i a_i^y s^3 + b_i^y s^2 + c_i^y s + d_i^y \quad (11)$$

The natural boundary condition sets the second derivative to zero at both endpoints: $x''(s_0) = x''(s_k) = 0$, ensuring the path does not curve artificially at start or goal.

The spline is then sampled at 300 evenly-spaced arc-length values to produce the final `smooth_path` list of `PathNode` objects published on `smooth_path`.

8 Pose Sources

`navigation_node` supports two ways to receive the robot's current position, selected by the `pose_source` parameter.

<code>pose_source</code>	Topic	Message	Use case
"dummy"	<code>current_position</code>	<code>PointStamped</code>	Testing with a fake publisher
"relocalization"	<code>/relocalization/pose</code>	<code>Pose</code>	Live ORB-SLAM3 output

8.1 Dummy Mode

In dummy mode the node subscribes to `current_position` (`geometry_msgs/PointStamped`). The message uses **user space** coordinates (Section 4): `point.x` is X_u (upward), `point.y` is Y_u (rightward), with origin at bottom-left.

A test publisher (`position_publisher.py`) shuttles a fake robot between two waypoints at 10 Hz to allow end-to-end testing without a robot.

8.2 Relocalization Mode

In relocalization mode the node subscribes to `/relocalization/pose` (`geometry_msgs/Pose`). The ORB-SLAM3 pose is in **world metres** and must be converted to pixel coordinates using the map metadata received on `/map_meta`:

Listing 2: World-to-pixel conversion in `relocalization__pose__callback`

```

1 col = int((msg.position.x - self._map_x_min) / self._map_resolution
2         )
3 row = map_h - 1 - int((msg.position.z - self._map_z_min) / self.
4         _map_resolution)
5 col = max(0, min(map_w - 1, col))
6 row = max(0, min(map_h - 1, row))
7 current = PixelCoords(col, row)

```

Common Mistake

Using raw world coordinates without subtracting $(x_{\min}, z_{\min})$ places the robot at pixel $(0,0)$ whenever it is near the ORB-SLAM3 world origin $(0,0,0)$ — which is almost certainly *outside* the map extent. The A* planner will immediately reject this as a blocked start cell. Always subtract the map offsets.

9 Deviation Detection and Replanning

The navigation node monitors whether the robot has strayed too far from the planned path. If it has, it replans automatically from the robot's current position.

Deviation Check

Given the current robot pixel position (r_x, r_y) and the set of smooth path nodes $\mathcal{P}$:

$$d_{\min} = \min_{p \in \mathcal{P}} \sqrt{(p_x - r_x)^2 + (p_y - r_y)^2}$$

If $d_{\min} > \tau$ (default $\tau = 5$ px), replanning is triggered.

A threshold of 5 pixels at 0.02 m/px corresponds to 10 cm in the real world. Replanning sets `start_coords` to the current robot position and calls `try_plan()` which re-runs A* and redraws the path image.

10 Visualization Pipeline

The navigation node maintains three image layers and publishes each as a separate ROS topic so different consumers can subscribe to the level of detail they need.

Topic	Contents	Updated when
<code>static_image</code>	Raw occupancy grid only	New map arrives
<code>path_image</code>	Grid + planned path + start/goal markers	New plan computed
<code>trails_image</code>	Path image + robot trail + heading indicator	Every pose update

10.1 Trail

The robot's position history is stored in a `deque(maxlen=500)`. When rendered, the trail is drawn as a grey polyline using `cv2.polylines`.

10.2 Heading Indicator (Triangle)

A cyan filled triangle indicates the robot's current heading. The triangle tip points in the direction of motion.

Heading Angle

The heading angle θ is computed from the robot's movement vector over a **10-frame lookback window**:

$$(dx, dy) = (x_{\text{now}} - x_{t-10}, y_{\text{now}} - y_{t-10})$$

$$\theta = \text{atan2}(dy, dx) \quad (\text{updated only if } dx^2 + dy^2 \geq 4)$$

The minimum-distance guard ($\|(dx, dy)\| \geq 2$ px) prevents single-pixel lateral noise from snapping the heading to left or right.

The triangle is defined in local coordinates and rotated by θ :

Triangle Rotation

Local triangle vertices (tip forward along $+x$):

$$V_{\text{local}} = \begin{bmatrix} L & 0 \\ -L/2 & -W \\ -L/2 & W \end{bmatrix} \quad (L = 9, W = 4)$$

Rotation matrix:

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

World vertices:

$$V = V_{\text{local}} R(\theta)^T + (c_x, c_y)$$

where (c_x, c_y) is the robot pixel position.

Note

The default heading at startup is $\theta = -\pi/2$ (pointing upward on screen). This means the triangle faces up even before the robot has accumulated enough trail to compute a direction.

11 Configuration

Runtime parameters are set in `src/navigation/navigation/config/navigation_params.yaml`:

Listing 3: `navigation__params.yaml`

```
navigation_node:
  ros__parameters:
```

```
pose_source: "dummy"      # "dummy" | "relocalization"
pose_scale:  1.0          # reserved; not used in current
coordinate math
```

Parameter	Default	Effect
pose_source	"dummy"	Subscribe to <code>current_position</code> (PointStamped) or <code>/relocalization/pos</code>
pose_scale	1.0	Multiplier reserved for future rescaling

After changing the YAML, rebuild once to propagate the file to the install tree:

```
colcon build --symlink-install --packages-select navigation
source install/setup.bash
```

12 End-to-End Flowchart

13 Key Takeaways

- 1. Inflate once, plan many times.** Obstacle inflation is the most expensive preprocessing step. By caching it after every map update and passing it directly to the planner, the system avoids re-inflating on every goal click or deviation replan.
- 2. Three coordinate systems, one internal representation.** All A* computation happens in pixel space (column, row). User clicks and ORB-SLAM3 poses are converted at the boundary, not inside the planner.
- 3. Map origin offset is not optional.** The occupancy grid shifts the world origin to its own bottom-left corner. Live poses *must* subtract $(x_{\min}, z_{\min})$ before dividing by resolution, or the robot will appear at a wrong location.
- 4. Lazy deletion makes A* fast on large grids.** Pushing a new heap entry instead of updating an existing one reduces the per-relaxation cost from $O(n)$ (linear scan) to $O(\log n)$. On a 200×200 grid this typically completes in under 5 ms.
- 5. Heading stability requires a look-back window.** Computing heading from just one previous trail point amplifies pixel-level noise. A 10-frame look-back with a 2-pixel minimum-movement guard produces a stable indicator that correctly reflects the robot's true direction of travel.

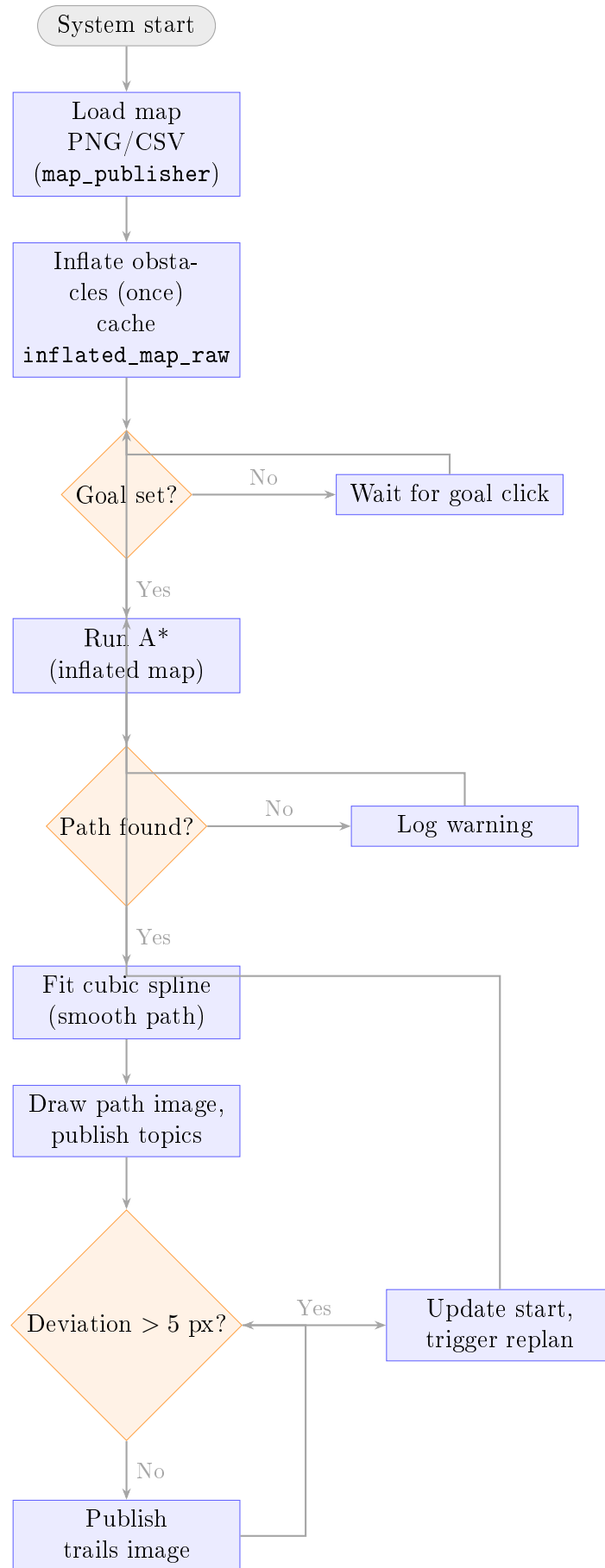


Figure 4: End-to-end planning and visualization loop.